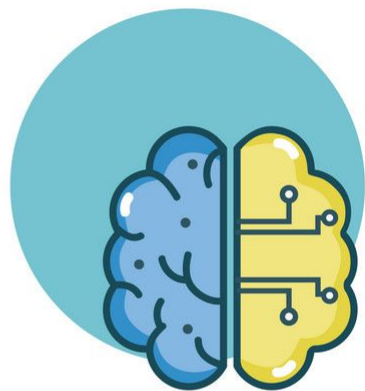


INTRODUCTION TO MACHINE LEARNING

K-NEAREST NEIGHBOR CLASSIFICATION



Elisa Ricci



DATA

Data



examples



REPRESENTING EXAMPLES



What is an example? How is it **represented**?

FEATURES

examples



features

$$x^1_1, x^1_2, x^1_3, \dots, \\ x^1_d$$

$$x^2_1, x^2_2, x^2_3, \dots, \\ x^2_d$$

$$x^3_1, x^3_2, x^3_3, \dots, \\ x^3_d$$

FEATURES

examples



features

grey, small, ...

brown, big, ...

black, small, ...

FEATURES

How our algorithms actually “view” the data

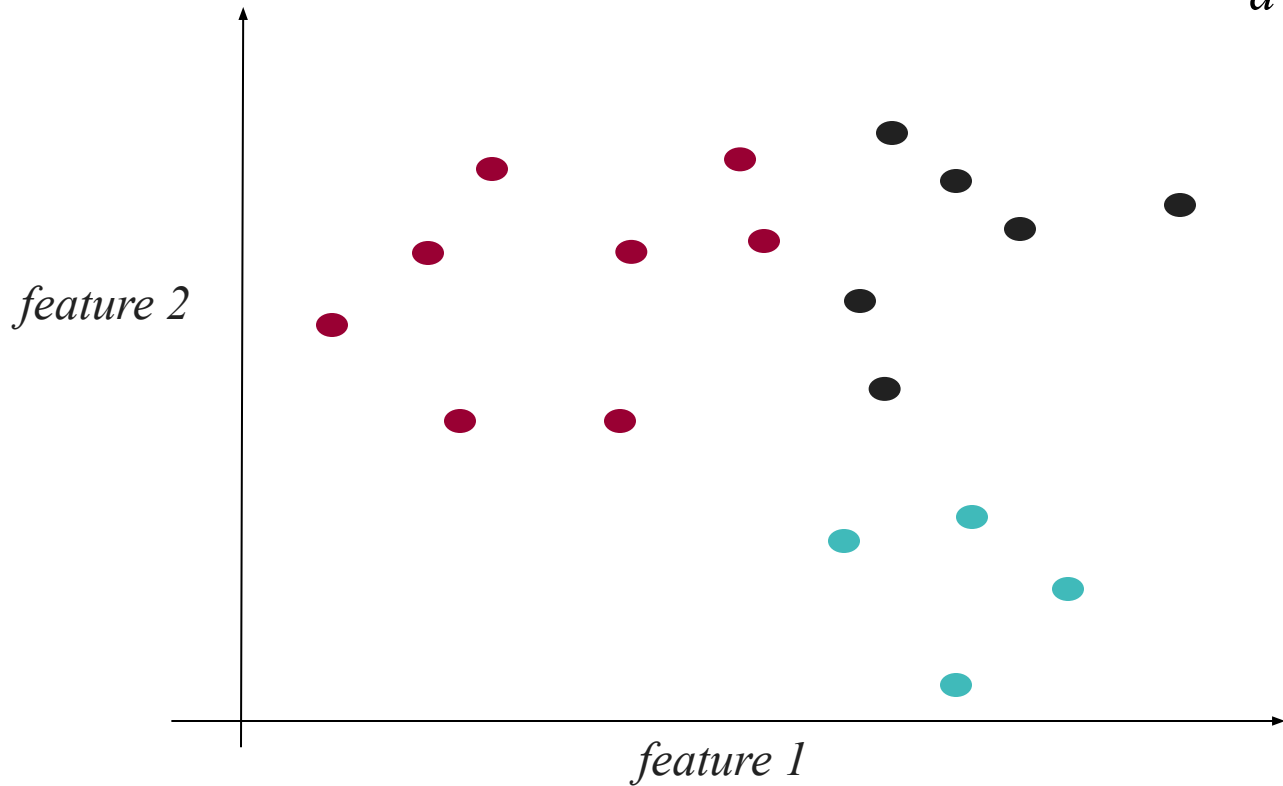
Features are the questions we can ask about the examples

Features in generally are **represented with** **vectors**

What does it imply?

EXAMPLES IN A FEATURE SPACE

Examples are points in an d -dimensional space where d is the number of features

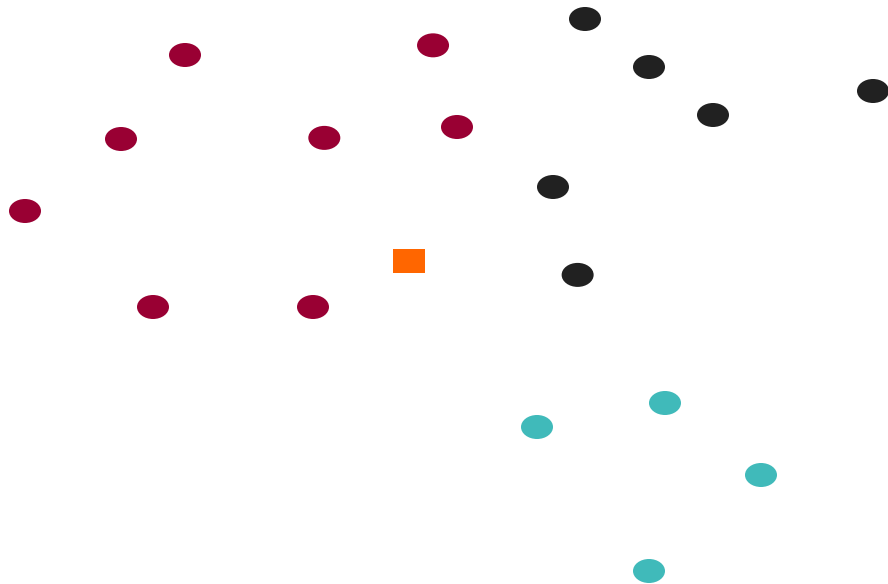


- *label 1*
- *label 2*
- *label 3*

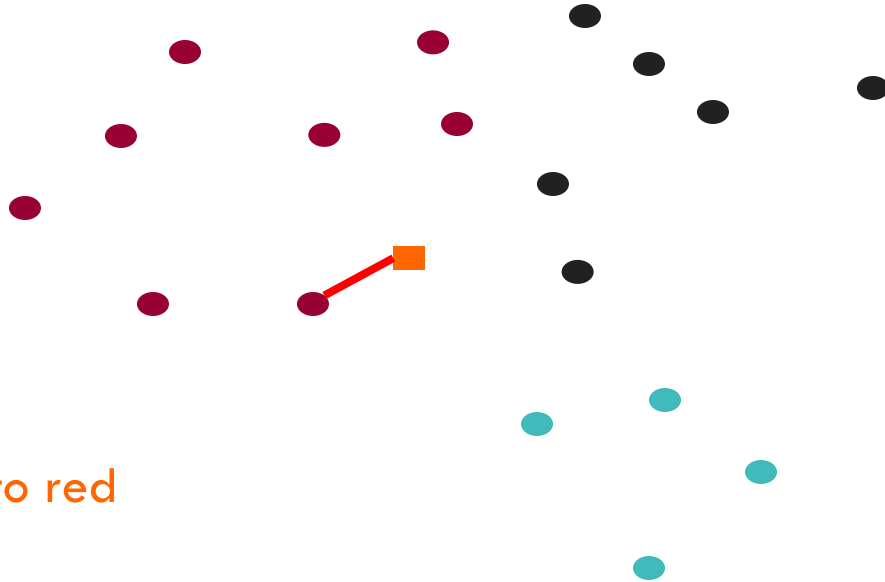
TRAINING SET



TEST EXAMPLE: WHICH CLASS?



TEST EXAMPLE: WHICH CLASS?

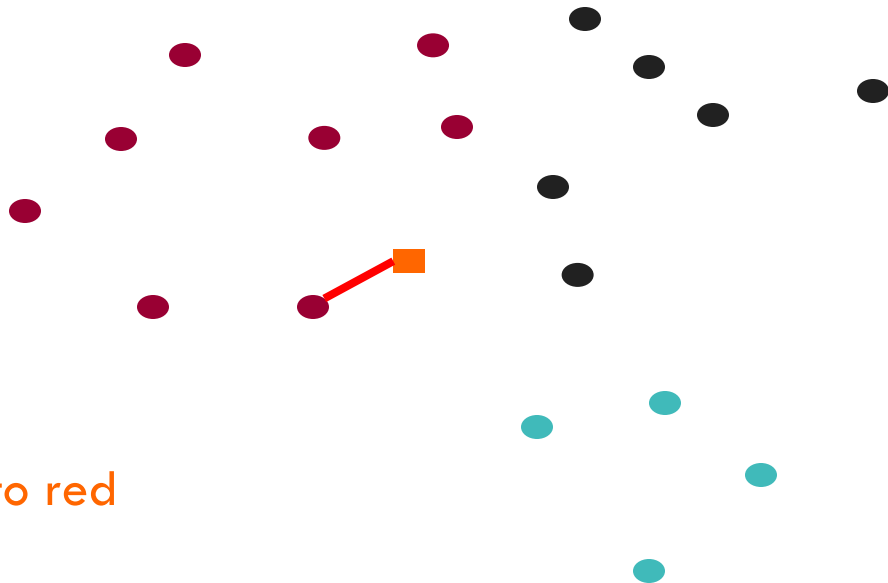


closest to red

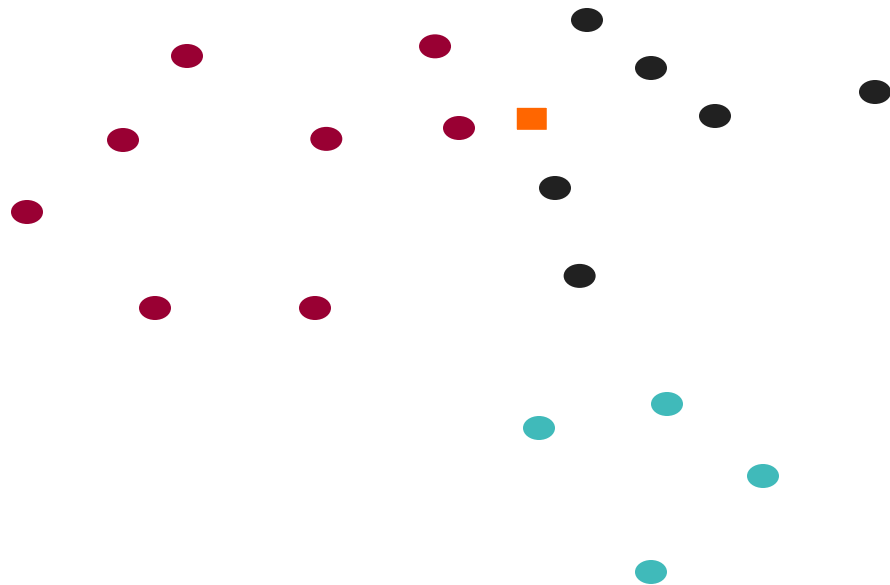
A CLASSIFICATION ALGORITHM?

To classify an example d :

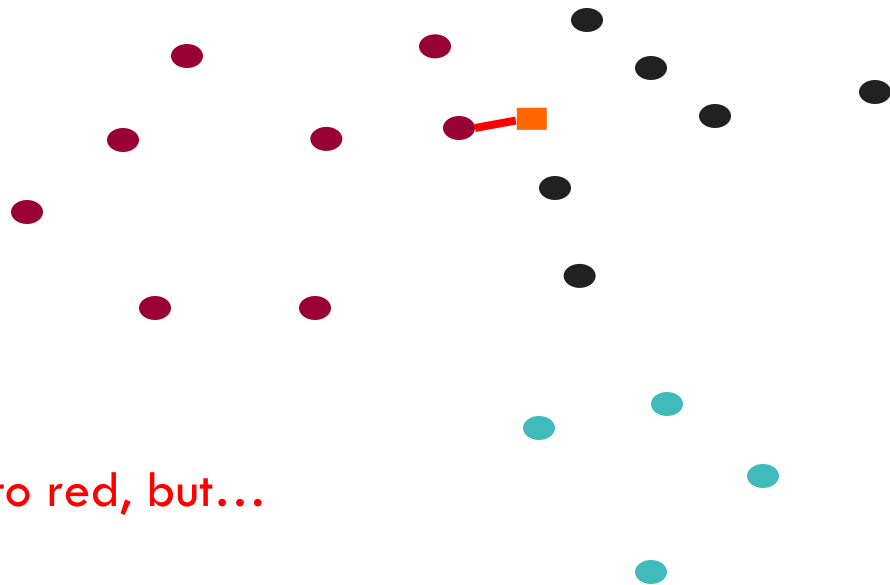
Label d with the label of the closest example to d in the training set



WHAT ABOUT THIS EXAMPLE?

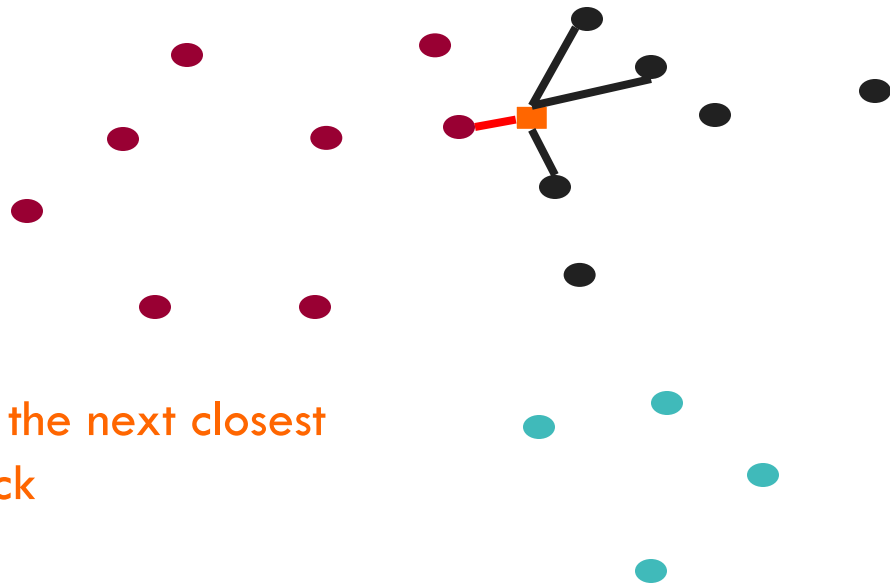


WHAT ABOUT THIS EXAMPLE?



closest to red, but...

WHAT ABOUT THIS EXAMPLE?



Most of the next closest
are black

K-NEAREST NEIGHBOR (K-NN)

To classify an example d :

- Find k nearest neighbors of d
- Choose as the label **the majority label** within the k nearest neighbors

Robust to noisy data by considering k -nearest neighbors

K-NEAREST NEIGHBOR (K-NN)

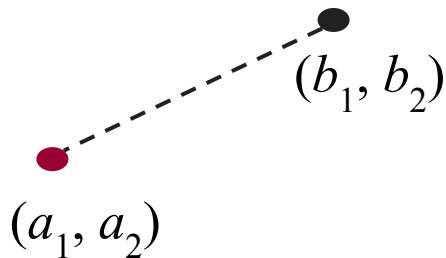
To classify an example d :

- Find k **nearest** neighbors of d
- Choose as the label **the majority label** within the k nearest neighbors

How do we measure “nearest”?

EUCLIDEAN DISTANCE

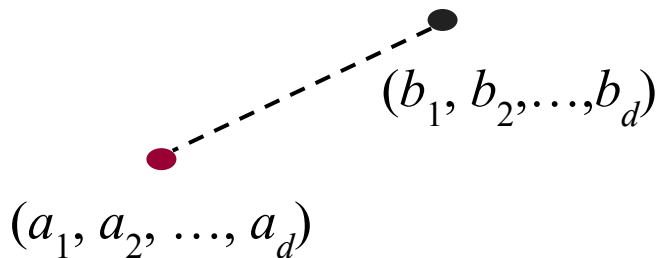
In two dimensions, how do we compute the distance?



$$D(\mathbf{a}, \mathbf{b}) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

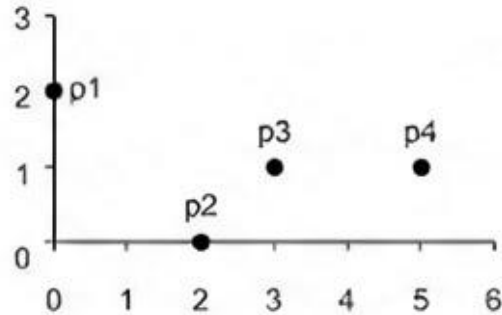
EUCLIDEAN DISTANCE

In d -dimensions, how do we compute the distance?



$$D(\mathbf{a}, \mathbf{b}) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_d - b_d)^2}$$

EUCLIDEAN DISTANCE: EXAMPLE



point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.882	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

EUCLIDEAN DISTANCE

- Euclidean Distance makes sense when different features are comparable (e.g., each variable measured in the same units).
 - For instance, if the measurements are different, say length and weight, it is not clear.
- When features are not comparable we can standardize them by dividing by their corresponding standard deviation or apply min-max scaling. This makes them all equally important.

STANDARDIZATION & SCALING

■ **Standardization or Z-score normalization**

- Rescale the data so that the mean is 0 and the standard deviation from the mean (standard scores) is 1

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

μ is mean, σ is a standard deviation from the mean
(standard score)

■ **Min-Max scaling**

- Scale the data to a fixed range, i.e. between 0 and 1

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

DISTANCE AND SIMILARITY

- Measuring distance or similarity is a domain-specific problem and there are many different variations
- **Similarity**
 - Numerical measure of how alike two data objects are.
 - Is higher when objects are more alike.
 - Often falls in the range $[0,1]$
- **Distance**
 - Numerical measure of how different are two data objects
 - Lower when objects are more alike
 - Minimum dissimilarity is often 0
 - Upper limit varies

MINKOWSKI DISTANCE

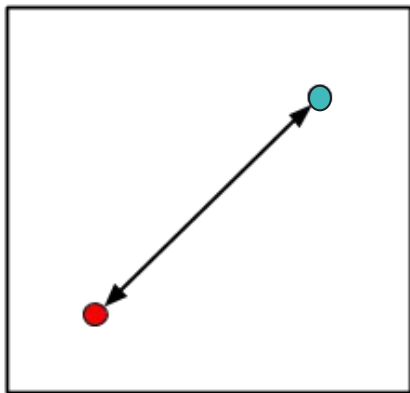
- Minkowski Distance is a generalization of Euclidean Distance

$$D(\mathbf{a}, \mathbf{b}) = \left(\sum_{i=1}^d |a_i - b_i|^r \right)^{1/r}, \quad r \geq 1$$

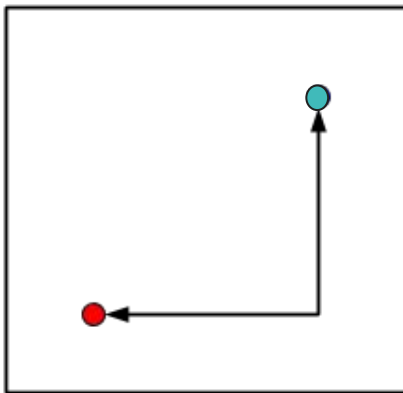
- r is a parameter, d is the number of dimensions
- $r = 1$. City block (Manhattan, L1 norm) distance.
- $r = 2$. Euclidean distance
- $r \rightarrow \infty$. “supremum” (L_∞ norm) distance. This is the maximum difference between any component of the vectors

DISTANCES

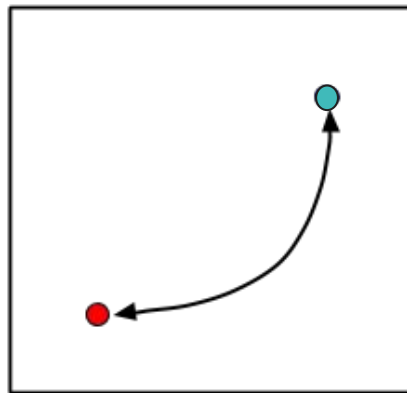
Euclidean



Manhattan



Minkowski

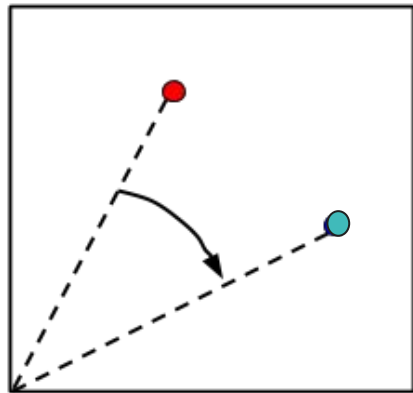


COSINE SIMILARITY

Given two vectors:

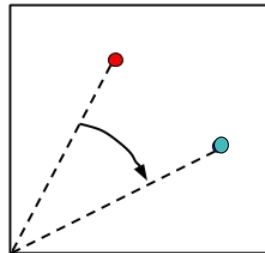
$$C(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Cosine similarity



where we divided the dot product among two vectors with their norms.

COSINE SIMILARITY



Geometric Interpretation

- Measures the cosine of the angle between two vectors.
- Focuses on the direction of vectors, disregarding their magnitude.
- Distance is obtained as $D(a,b)=1-C(a,b)$

Values:

- **1**: Vectors are perfectly aligned (maximum similarity).
- **0**: Vectors are orthogonal (no similarity).
- **-1**: Vectors are in opposite directions.

Applications:

- Widely used in applications such as computing document similarity.

DECISION BOUNDARIES

The **decision boundaries** are places in the features space where the classification of an example changes



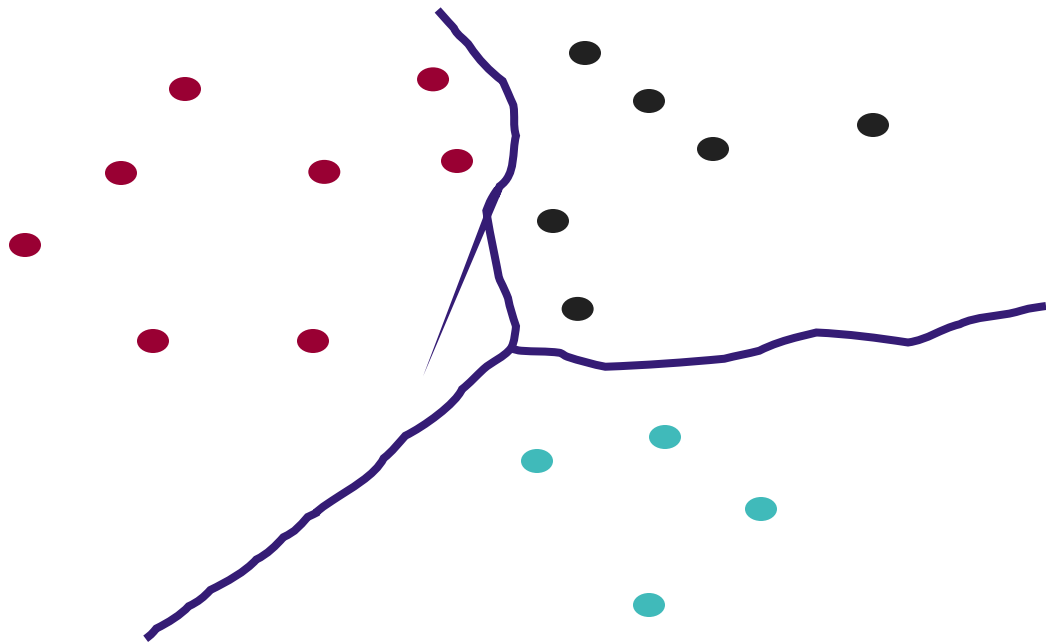
DECISION BOUNDARIES

Where are the decision boundaries for k-NN?



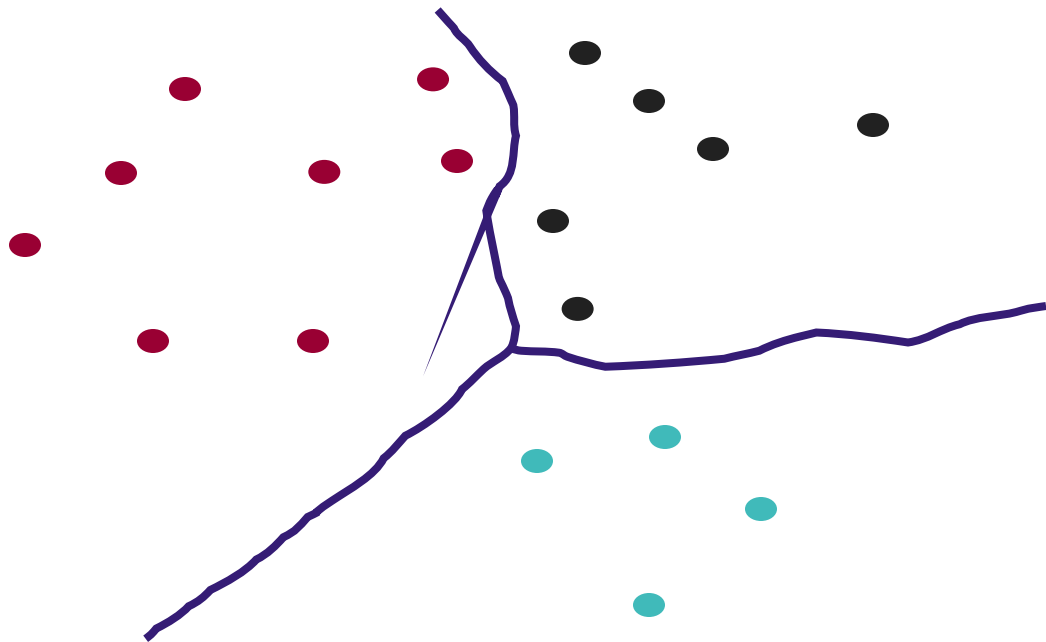
DECISION BOUNDARIES

Where are the decision boundaries for k-NN?



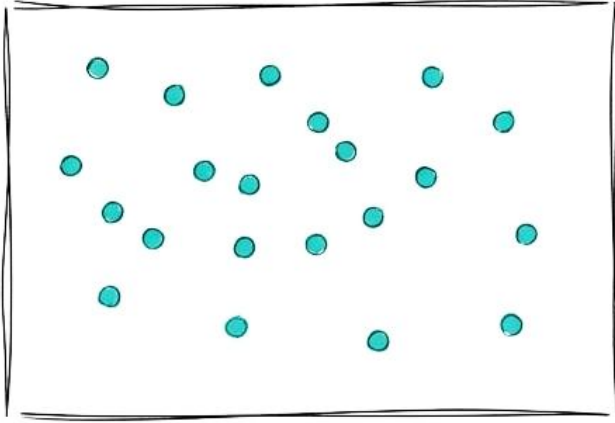
DECISION BOUNDARIES

k-NN gives **locally defined decision boundaries** between classes

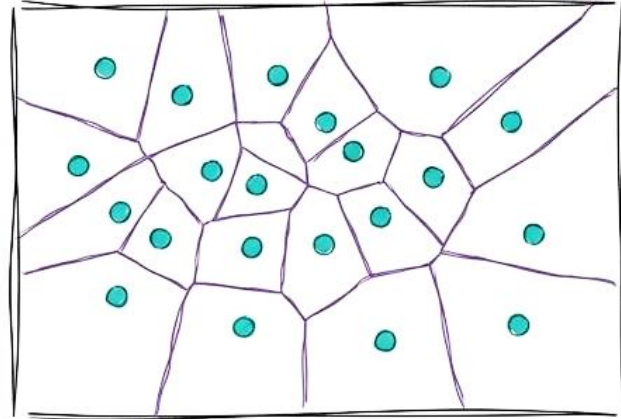


VORONOI DIAGRAM

- Describes the areas that are nearest to any given point, given a set of data.
- Each line segment is equidistant between two points



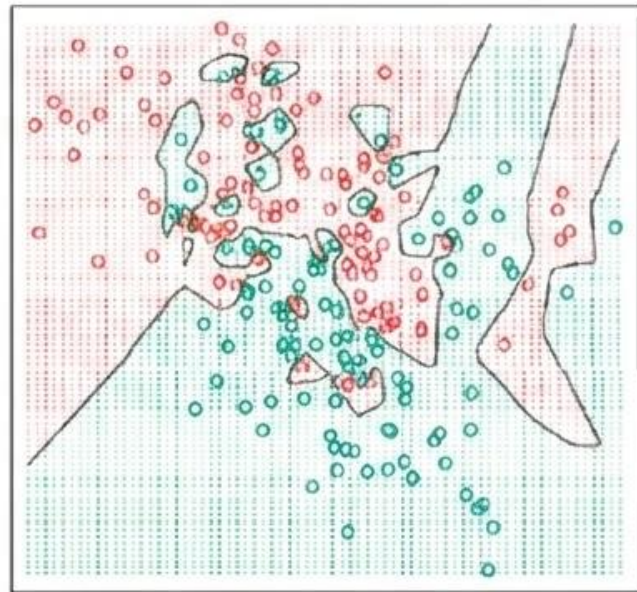
Initial points



Voronoi Diagram

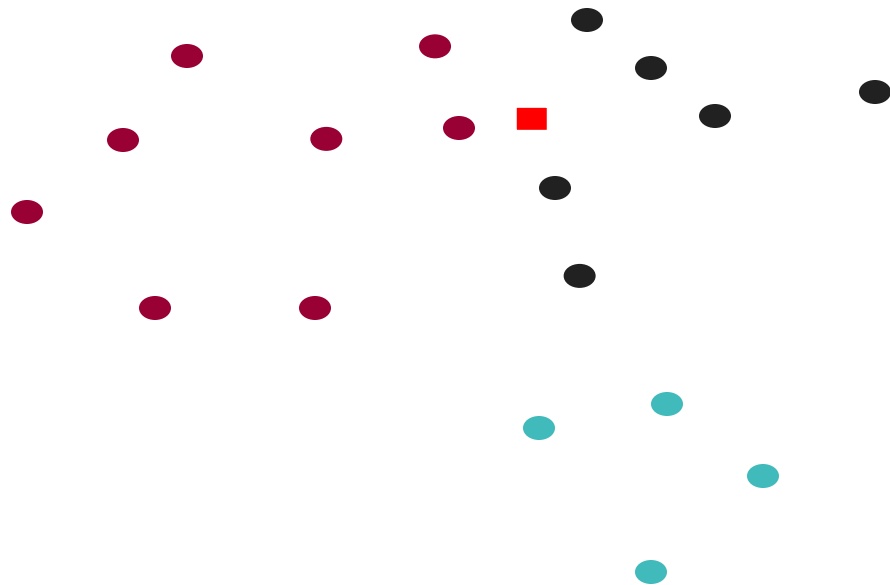
DECISION BOUNDARIES

- k-NN algorithm does **not explicitly** compute decision boundaries.
- The decision boundaries form a **subset of the Voronoi diagram** for the training data.
- The more examples that are stored, the more complex the decision boundaries can become



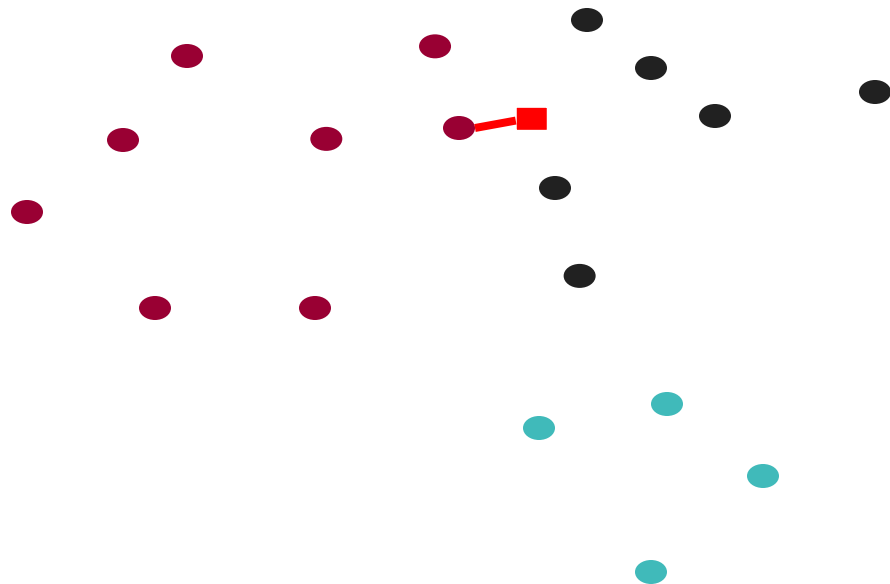
CHOOSING K

What is the label with $k = 1$?



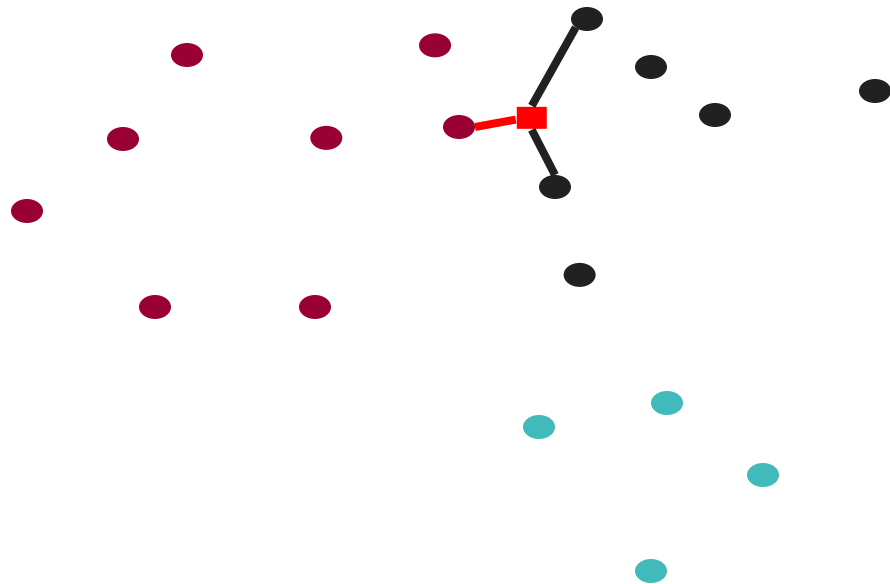
CHOOSING K

We choose red. Do you agree?



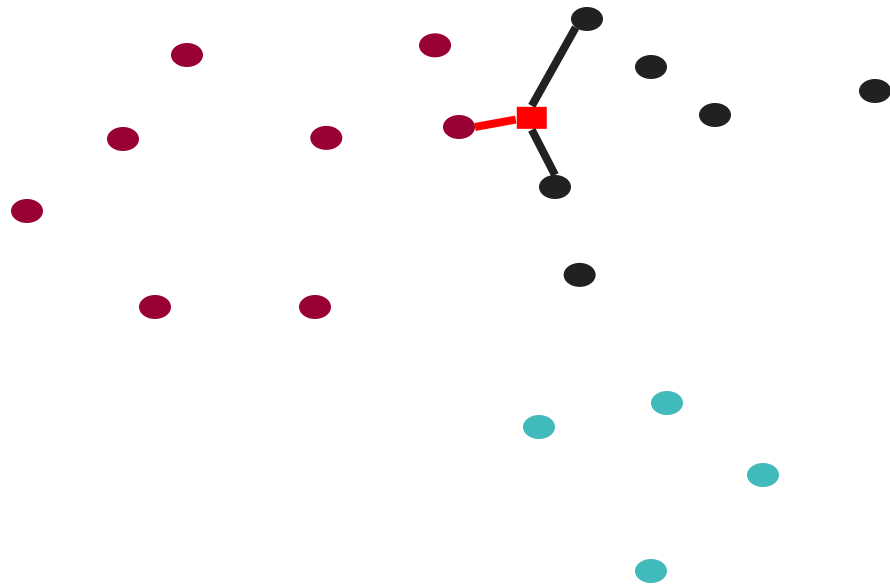
CHOOSING K

What is the label with $k = 3$?



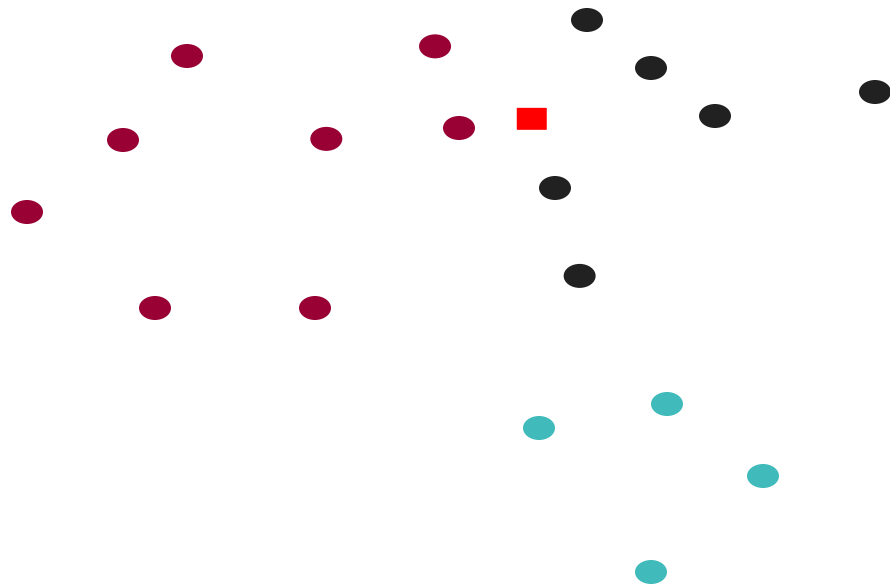
CHOOSING K

We choose black. Do you agree?



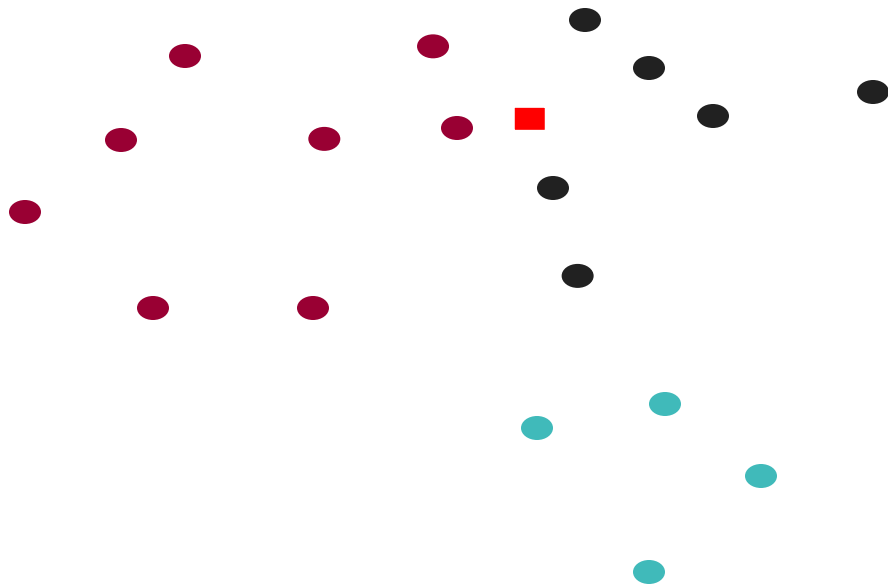
CHOOSING K

What is the label with $k = 20$?



CHOOSING K

What is the label with $k = 20$? Red! (class with max number of datapoints, weird)

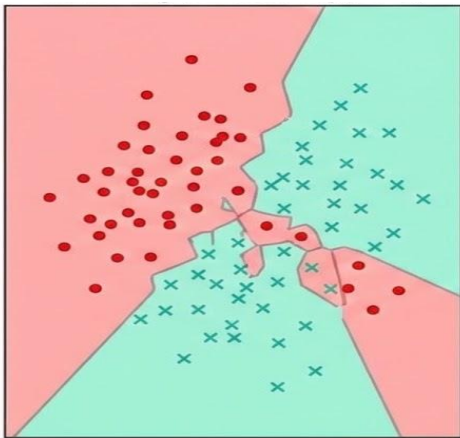


THE IMPACT OF K

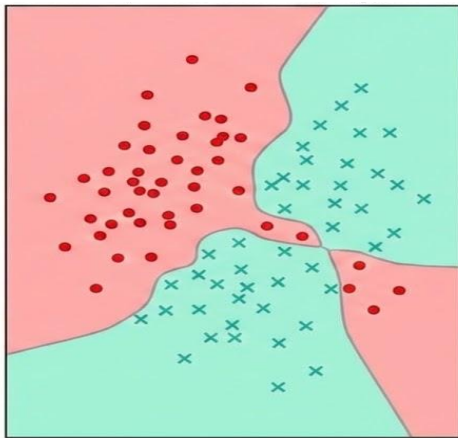
What is the role of k ?

How does it relate to overfitting and underfitting?

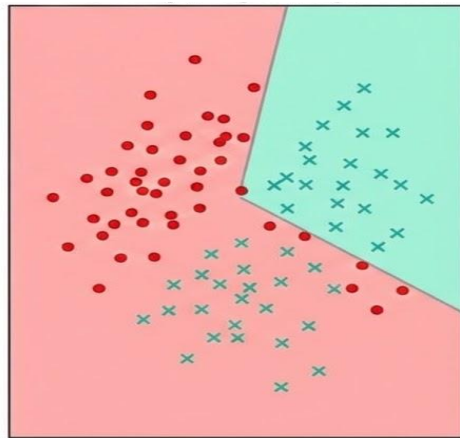
Overfitting ($k=1$)



Ideal



Underfitting ($k=35$)



THE IMPACT OF k

- A larger value of k results in a smoother decision boundary.
- If k equals the total number of data points (N), the prediction will always be the majority class.
- Small k (e.g., $k = 1$) corresponds to overfitting
 - Low training error, very sensitive to noise.
- Large k corresponds to underfitting
 - Higher training error, overly smooth decision boundary.
- Trade-off: choose k that minimizes validation/test error.

RECAP: UNDERFITTING & OVERFITTING

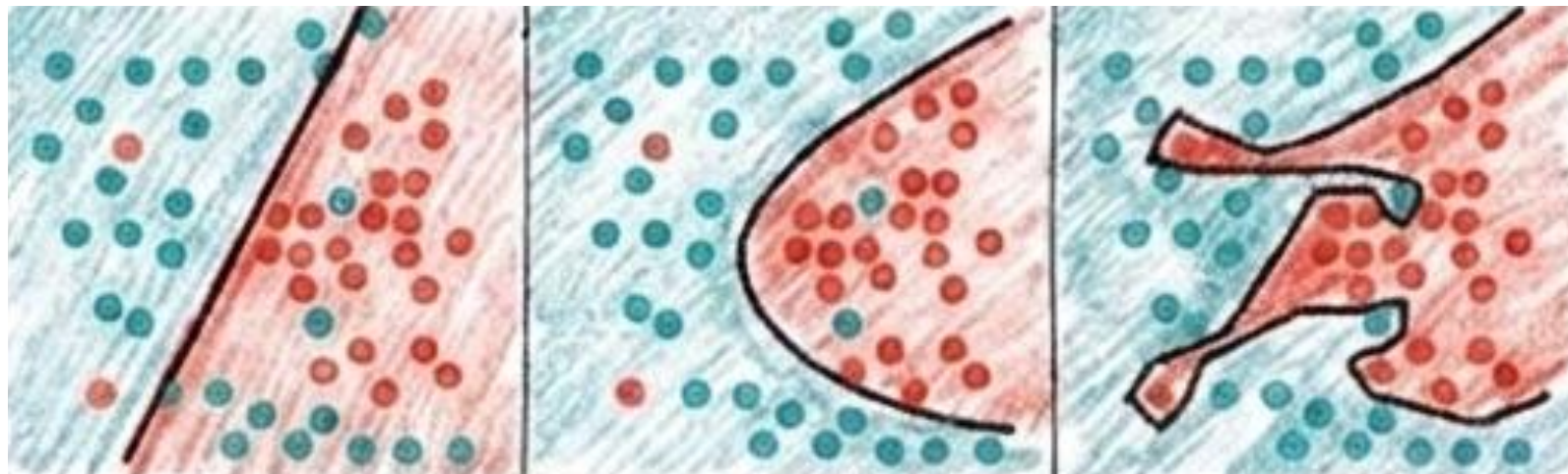
- The factors determining how well a machine learning algorithm will perform are its ability to:
 - Make the **training error small**.
 - Make the **gap between training and test error small**.
- These two factors correspond to the cases of **underfitting** and **overfitting**.
- **Underfitting** occurs when the model is not able to obtain a sufficiently low error value on the training set.
- **Overfitting** occurs when the gap between the training error and test error is too large.

RECAP: UNDERFITTING & OVERFITTING

Underfitting

Ideal

Overfitting



Classification

K-NEAREST NEIGHBOR (K-NN)

To classify an example d :

- Find k nearest neighbors of d
- Choose as the label the majority label within the k nearest neighbors

How do we choose k ?

HOW TO PICK K

- Common heuristics:
 - often 3, 5, 7
 - choose an odd number to avoid ties
- Use validation set or use cross-validation
- Rule of thumb is $k < \sqrt{N}$, where N is the number of training examples

AN ASIDE: CROSS-VALIDATION

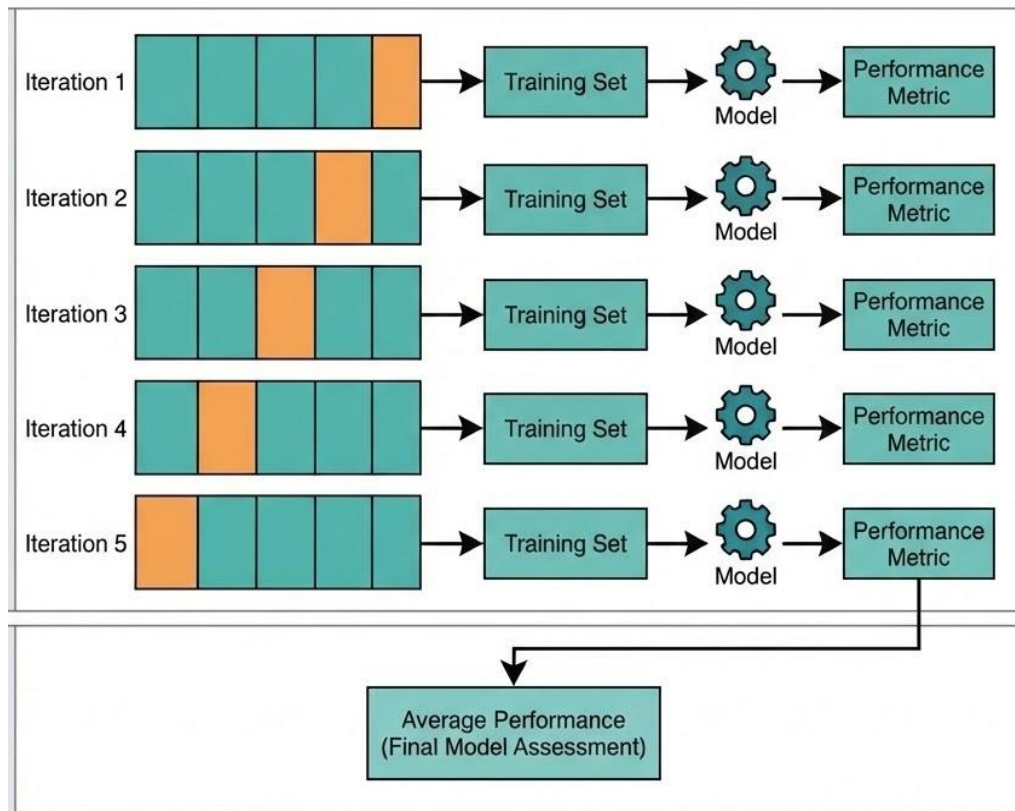
A resampling method used to estimate generalization performance and select model hyperparameters.

PROCEDURE

- Split the dataset into m **equal folds**.
- For each fold:
 - Train the model on $m - 1$ **folds**
 - Validate on the remaining fold
 - Repeat m **times**, each fold used once as validation.
- Compute the **average validation error** across folds.

AN ASIDE: CROSS-VALIDATION

- CV provides a more reliable estimate of test error
- It helps select model complexity (e.g., k in kNN, regularization parameter)



K-NEAREST NEIGHBOR (K-NN)

To classify an example d :

- Find k **nearest** neighbors of d
- Choose as the label **the majority label** within the k nearest neighbors

Any other idea?

K-NN VARIATIONS

Instead of giving equal importance to the k neighbors, assign larger weights to closer neighbors. It reduces influence of distant neighbors but still simple to implement

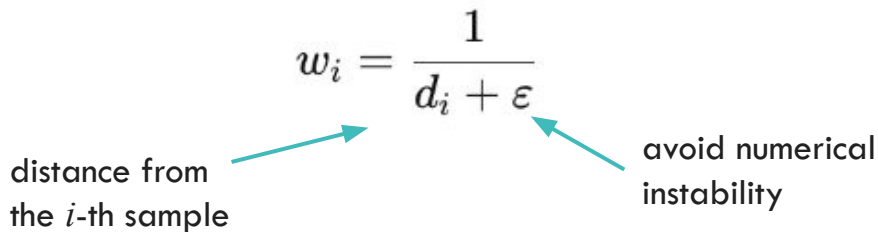
Weighted k -NN:

- Right now, all examples are treated equally
- Weight the “vote” o o that closer examples have higher weight

$$w_i = \frac{1}{d_i + \epsilon}$$

distance from the i -th sample

avoid numerical instability

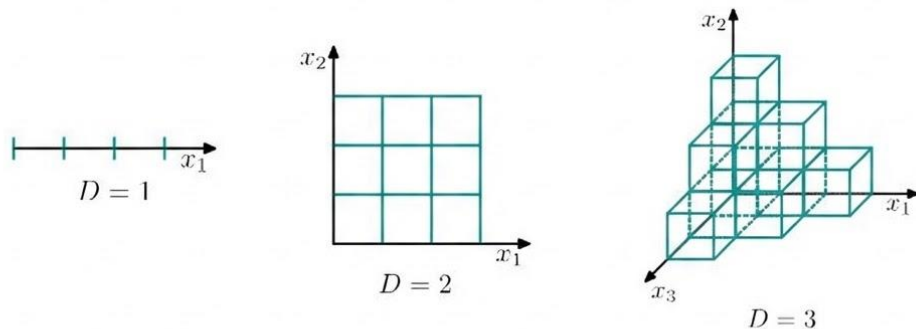
A diagram showing the formula $w_i = \frac{1}{d_i + \epsilon}$. A teal arrow points from the text "distance from the i -th sample" to the d_i term in the denominator. Another teal arrow points from the text "avoid numerical instability" to the ϵ term in the denominator.

LAZY LEARNER VS EAGER LEARNER

- k-NN is belongs to the class of **lazy learning** algorithms
- **Lazy learning**: Simply stores training data (or perform limited processing) and operates when it is given a test example
- **Eager learning** (e.g. Decision Trees, SVMs): Given a training set, constructs a classification model before receiving new test data to classify
- Lazy learners: less time in training but more time in predicting

CURSE OF DIMENSIONALITY

- **Curse of dimensionality:** In high dimensions almost all points are far away from each other
- The size of the data space grows exponentially with the number of dimensions.
- This means that the size of the data set must also grow exponentially in order to keep the same density.



CURSE OF DIMENSIONALITY

- The success of k-NN is very dependent on having a **dense data set.**
- Every machine learning algorithm needs a dense data set for accurate prediction
- What makes k-nearest neighbors special?
- k-NN requires a point to be close in **every single dimension.**
- Some algorithms can create models based on single dimensions and only need points to be close together along that axis.
- k-NN does not work that way. It needs all points to be close along every axis in the data space.

CURSE OF DIMENSIONALITY: SUMMARY

- The problem is fundamentally that there is not enough data available for the number of dimensions.
- As the number of dimensions increases the size of the data space increases and the amount of data needed to maintain density also increases.
- Without dramatic increases in the size of the data set, k-NN loses all predictive power.

COMPUTATIONAL COST

- Linear algorithm (no pre-processing):
 - **Training cost:** very low but memory cost: $O(dN)$
 - **Inference** (per point): compute distance for all N datapoints; complexity of distance computation: $O(dN)$
- Prediction is expensive for large datasets and becomes slow in high dimensions
- It can be accelerated with KD-trees & Ball trees or approximate nearest neighbor (ANN) methods

AN ASIDE: PARAMETRIC VS NON-PARAMETRIC MODELS

- **Parametric models:** we have a *finite number of parameters*
 - Linear regression, logistic regression and linear Support Vector Machines
- **Nonparametric models:** the number of parameters is (potentially) infinite. The *complexity of the model grows with the number of training data*.
 - K-NN, decision trees or non-linear SVMs are considered as non-parametric learning algorithms since the number of parameters grows with the size of the training set.

K-NEAREST NEIGHBOR: SUMMARY

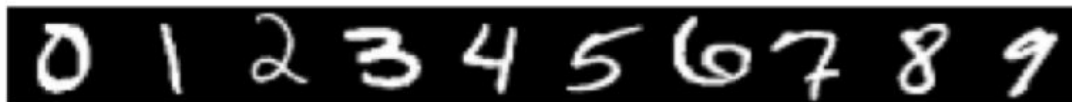
- **Non parametric** model
- When to consider
 - Few (e.g. less than 20) features per instance
 - Lots of training data
- Advantages
 - Training is very fast (basically no training)
 - Learn complex target functions
- Disadvantages
 - Slow at inference time
 - Easily fooled by irrelevant attributes

]

K-NEAREST NEIGHBOR: ADVANTAGES

- Easy to program
- No optimization or training required
- Classification accuracy can be very good, it can outperform more complex models

EXAMPLE



- Yann LeCunn – MNIST Digit Recognition
 - Handwritten digits
 - 28x28 pixel images: $d = 784$
 - 60,000 training samples
 - 10,000 test samples
- Nearest neighbour is competitive

	Test Error Rate (%)
Linear classifier (1-layer NN)	12.0
K-nearest-neighbors, Euclidean	5.0
K-nearest-neighbors, Euclidean, deskewed	2.4
K-NN, Tangent Distance, 16x16	1.1
K-NN, shape context matching	0.67
1000 RBF + linear classifier	3.6
SVM deg 4 polynomial	1.1
2-layer NN, 300 hidden units	4.7
2-layer NN, 300 HU, [deskewing]	1.6
LeNet-5, [distortions]	0.8
Boosted LeNet-4, [distortions]	0.7

K-NEAREST NEIGHBOR: ISSUES

- Choose distance measure
 - Most common: Euclidean
- Choose k
- Curse of Dimensionality
- Memory based technique
 - Must make a pass through the data for each classification. This can be prohibitive for large data sets.

REAL WORLD EXAMPLE

Problem: Where (e.g., which country or GPS location) was this picture taken?



REAL WORLD EXAMPLE

Problem: Where (e.g., which country or GPS location) was this picture taken?

IM2GPS: estimating geographic information from a single image

James Hays and Alexei A. Efros
Carnegie Mellon University

Abstract

Estimating geographic information from an image is an excellent, difficult high-level computer vision problem whose time has come. The emergence of vast amounts of geographically-calibrated image data is a great reason for computer vision to start looking globally – on the scale of the entire planet! In this paper, we propose a simple algorithm for estimating a distribution over geographic locations from a single image using a purely data-driven scene matching approach. For this task, we will leverage a dataset of over 6 million GPS-tagged images from the Internet. We represent the estimated image location as a probability distribution over the Earth's surface. We quantitatively evaluate our approach in several geolocation tasks and demonstrate encouraging performance (up to 30 times better than chance). We show that geolocation estimates can provide the basis for numerous other image understanding tasks such as population density estimation, land cover estimation or urban/rural classification.

1. Introduction

Consider the photographs in Figure 1. What can you say about where they were taken? The first one is easy – it's an iconic image of the Notre Dame cathedral in Paris. The middle photo looks vaguely Mediterranean, perhaps a small town in Italy, or France, or Spain. The rightmost photograph is the most ambiguous. Probably all that could be said is that it's a picture of a seaside in some tropical location. But note that even this vague description allows us to disregard all non-coastal, non-tropical areas – more than 99.9% of the Earth's surface! Evidently, we humans have learned a reasonably strong model for inferring location distribution from photographs. Moreover, even in cases when our geo-localization performance is poor, we are still able to give fairly confident estimates to other related questions: How hot/cold does it get? How many people live there? How well-off are they? etc.

What explains this impressive human ability? Semantic reasoning, for one, is likely to play a big role. People's faces and clothes, the language of the street signs, the types of trees and plants, the topographical features of the terrain – all can serve as semantic clues to the geographic location of a particular shot. Yet, there is mounting evidence in cognitive science that *data association* (ask not “What is it?” but rather “What is it *like*?”) may play a significant role as well [1]. In the example above, this would mean that instead of reasoning about a beach scene in terms of the trop-



Figure 1. What can you say about where these photos were taken?

ical sea, sand and palm trees, we would simply remember: “I have seen something similar on a trip to Hawaii!”. Note that although the original picture is unlikely to actually be from Hawaii, this association is still extremely valuable in helping to implicitly define the type of place that the photo belongs to.

Of course, computationally we are quite far from being able to semantically reason about a photograph (although encouraging progress is being made). On the other hand, the recent availability of truly gigantic image collections has made data association, such as brute-force scene matching, quite feasible [17, 4].

In this paper, we propose an algorithm for estimating a distribution over geographic locations from an image using a purely data-driven scene matching approach. For this task, we leverage a dataset of over 6 million GPS-tagged images from the Flickr online photo collection. We represent the estimated image location as a probability distribution over the Earth's surface, and geolocation performance is analyzed in several tasks. Additionally, the usefulness of image localization is demonstrated with meta-tasks such as land cover estimation and urban/rural classification.

1.1. Background

Visual localization on a topographical map has been one of the early problems in computer vision, which turned out to be extremely challenging for both computers and humans [16]. But the situation improves dramatically if more sources of data are available. Jacobs et al. [6] proposes a very clever and simple method of geolocating a webcam based on correlating its video-stream with satellite weather maps over the same time period.

The recent availability of GPS-tagged images of urban environments coupled with advances in multi-view geometry and efficient feature matching led to a number of groups developing place recognition algorithms, some of which competed in the “Where am I?” Contest [15] at ICCV’05 (winning entry described in [19]). Similar feature-based geometric matching approaches have also been successfully applied to co-registering online photographs of famous landmarks for browsing [14] and summarization [13],

¹Project Page: <http://graphics.cs.cmu.edu/projects/im2gps/>

REAL WORLD EXAMPLE

- Get 6M images from Flickr with gps info (dense sampling across world)
- Represent each image with meaningful features
- Do kNN (large k better, they use $k = 120$)



<http://graphics.cs.cmu.edu/projects/im2gps/>

<http://graphics.cs.cmu.edu/projects/im2gps/results.html>

<https://www.youtube.com/watch?v=dqif39IKT9A>

QUESTIONS?

